PS Funktionale Programmierung, SS 2024 – Blatt 7

Note: First go through the code from the lecture and be sure to understand how streams work and how we managed to implement this powerful concept ourselves!

DO NOT USE Racket-streams. Use the self-made s-xxx procedures.

Copy&Paste the required code from the lecture into your homework file.

1.

a) Implement **s-length** that returns the length of a stream.

b) Use *s-map* to define a procedure **toCelsius-stream** that takes a stream of temperatures in Fahrenheit and returns a stream of corresponding temperatures in Celsius, using the formula C = 5/9 (F – 32). Process the stream in two stages (i.e., produce an intermediate stream!): first subtract 32 from each temperature then multiply the results by 5/9.

2. Define two procedures **list2s** and **s2list** to convert from a list to a stream and vice versa.

3. Define an infinite stream of powers of two starting from a certain number.
Simplification: You are allowed to assume that the given number is also a power of 2.
E.g., `(powersOf2 64)` → `stream: 64,128,256,512,1024,…`

4. Define a procedure **(s-add s1 s2)** that creates a new stream by performing element-wise addition of *s1* and *s2*.
E.g., `(s-add integers integers)` → `stream: 2, 4, 6, 8, …`

5. Implement the *iterative improve* example from Lecture 1 **with streams** to calculate the square root (SICP, chapter 3.5.3). Note: do not use the code shown in the lecture, make it simpler (without using map).

a) Write a function **sqrt-stream** that shall return a stream of improving guesses g_0, g_1, …..

E.g., `(sqrt-stream 2)` → `stream: 1.0, 1.5, 1.4166666666666665, 1.4142156862745097, …`

b) Write a function **(sqrt x epsilon)** that is based on sqrt-stream and returns the first guess that is "good enough" (the absolute error is smaller or equal than the specified epsilon).
E.g. `(sqrt 2 0.1)` → `1.4166666666666665`